

CO3 – AT2

RAG BASED SYSTEMS

Top-K Retrieval Demonstration and Library Information RAG System

**Made By
G.Vinay Kumar Reddy**

Retrieval-Augmented Generation (RAG)

A practical demonstration of Top-1, Top-3, Top-5 retrieval
and a library information question-answering system

17. TOP-K RETRIEVAL DEMONSTRATION

Retrieval-Augmented Generation (RAG)

Objective

Develop a RAG system where the user can select Top-1, Top-3, or Top-5 retrieval and compare the retrieved context.

Introduction

Retrieval-Augmented Generation is an approach in which relevant information is retrieved from a collection of documents before an answer is produced. Instead of depending only on information stored in a model, the system first searches its available knowledge and uses the retrieved text as context. This makes the answer more closely connected to the information supplied to the system.

The Top-K value controls how many relevant documents are retrieved. Top-1 returns one document, Top-3 returns three documents, and Top-5 returns five documents. The demonstration below uses a simple word-overlap score so that the complete program remains short and can be run directly in Python IDLE or VS Code.

Top-K Retrieval: Working and Comparison

Working Principle

The system begins with a small document collection. When the user enters a query, the query is divided into words. Each document is also divided into words. The number of common words is used as a simple relevance score. Documents with higher scores are placed before documents with lower scores. The selected K documents are then displayed as the retrieved context.

Top-K Selection

K	Retrieved Information	Advantage	Limitation
Top-1	One best matching document	Focused context	May miss supporting information
Top-3	Three best matching documents	Balanced context	Some extra information may appear
Top-5	Five best matching documents	Maximum coverage	Context can become less focused

Comparison

Top-1 is useful when the query has a clear and direct answer. Top-3 provides a broader context and can combine information from several related documents. Top-5 provides the largest context in this demonstration and is useful when the query requires information from multiple documents. However, increasing K does not always mean a better answer because unrelated information can also be retrieved.

RAG Context

After retrieval, the selected documents can be combined into one context. A generation component can then use this context to form the final response. In the present small implementation, the combined context is displayed instead of using an external generative model.

Top-K Retrieval – Python Implementation

Python Code

```
documents = [
    "RAG combines retrieval with generation using context.",
    "Top-K retrieval selects the K most relevant documents.",
    "Python is used for artificial intelligence and data science.",
    "Natural Language Processing works with human language.",
    "Vector databases store and retrieve document embeddings."
]

def score(query, document):
    q = set(query.lower().split())
    d = set(document.lower().split())
    return len(q & d)

query = input("Enter your question: ")
k = int(input("Enter Top-K (1, 3 or 5): "))

if k not in [1, 3, 5]:
    print("Invalid choice. Select 1, 3 or 5.")
else:
    ranked = sorted(
        documents,
        key=lambda x: score(query, x),
        reverse=True
    )

    print("\nRetrieved Context:")
    for i, text in enumerate(ranked[:k], 1):
        print(i, ":", text)

    print("\nCombined Context:")
```

```
print(" ".join(ranked[:k]))
```

Code Explanation

The documents list represents the retrieval database. The score() function calculates the number of common words between the query and a document. The input statement accepts the user's question and K value. The sorted() function ranks the documents according to their scores. Finally, ranked[:k] selects the required number of documents.

Expected Output

Enter your question: What is Top-K retrieval?

Enter Top-K (1, 3 or 5): 3

Retrieved Context:

- 1 : Top-K retrieval selects the K most relevant documents.
- 2 : RAG combines retrieval with generation using context.
- 3 : Vector databases store and retrieve document embeddings.

Top-K Retrieval – Result and Analysis

Result

The program allows the user to dynamically select Top-1, Top-3, or Top-5 retrieval. The output changes according to the selected value of K. This demonstrates the retrieval stage of a RAG pipeline and clearly shows how the amount of context changes when K is increased.

Analysis

For Top-1, the system returns only the highest-ranked document, which produces a compact context. For Top-3, the system returns three documents and therefore has more information available for answering a query. For Top-5, the system retrieves the largest available context in the example. The comparison demonstrates the trade-off between focused retrieval and information coverage.

Example of Context Comparison

Top-1: Only the most relevant statement is supplied. This is concise and easy to process.

Top-3: Three related statements are supplied, giving the system additional supporting information.

Top-5: Five statements are supplied, giving the widest coverage but potentially including less relevant text.

Conclusion

The Top-K demonstration successfully shows the retrieval component of RAG. It provides a simple and understandable comparison between Top-1, Top-3, and Top-5 retrieval without requiring external libraries.

18. LIBRARY INFORMATION RAG SYSTEM

Objective

Create a RAG system using library rules that answers questions about borrowing, renewal, fines, timings, and membership.

Introduction

A Library Information RAG System is a practical application of retrieval-based question answering. Library rules are stored as a small knowledge base. When a student asks a question, the system searches the rules and retrieves the rule that best matches the question. The retrieved rule is then displayed as the context and answer.

The implementation is intentionally compact. It does not require a database or external NLP framework. The matching process uses common words between the user's question and the library rules. This makes the RAG concept easy to demonstrate and test using a normal Python installation.

Library Rules

Topic	Rule
Borrowing	Students can borrow up to 4 books for 14 days.
Renewal	A book can be renewed once for another 7 days if it is not reserved.
Fines	The late fine is Rs. 2 per book for each overdue day.
Timings	The library is open Monday to Saturday from 9:00 AM to 6:00 PM.
Membership	Students need a valid student ID. Membership is free.

Library Information RAG – Python Implementation

Python Code

```
rules = [
    ("Borrowing",
     "Students can borrow up to 4 books for 14 days."),
    ("Renewal",
     "A book can be renewed once for another 7 days if it is not reserved."),
    ("Fines",
     "The late fine is Rs. 2 per book for each overdue day."),
    ("Timings",
     "The library is open Monday to Saturday from 9:00 AM to 6:00 PM."),
    ("Membership",
     "Students need a valid student ID. Membership is free.")
]

def retrieve(question):
    q = set(question.lower().split())
    results = []

    for topic, text in rules:
        words = set((topic + " " + text).lower().split())
        score = len(q & words)
        results.append((score, topic, text))

    results.sort(reverse=True)
    return results[0]

question = input("Ask a library question: ")
score, topic, answer = retrieve(question)

if score == 0:
    print("No relevant library information found.")
else:
    print("\nRetrieved Context:")
    print(topic + ": " + answer)
    print("\nAnswer:")
    print(answer)
```

Code Explanation

The rules list stores the library knowledge base. Each rule contains a topic and its corresponding information. The retrieve() function converts the question into a set of words and compares it with every rule. The score represents the number of matching words. The rule with the highest score is selected and returned. If no words match, the program informs the user that no relevant information was found.

Retrieved Context

The retrieved topic and rule form the context for the answer. For example, a question containing words such as 'late', 'fine', or 'overdue' can match the Fines rule. A question containing 'renew' can match the Renewal rule.

Library Information RAG – Questions and Answers

Sample Questions and Answers

Question: How many books can I borrow?

Answer: Students can borrow up to 4 books for 14 days.

Question: How long can I keep a book?

Answer: The normal borrowing period is 14 days.

Question: Can I renew a book?

Answer: A book can be renewed once for another 7 days if it is not reserved.

Question: How many times can I renew?

Answer: A book can be renewed once.

Question: What happens if the book is reserved?

Answer: The book cannot be renewed if it is reserved by another member.

Question: What is the late fine?

Answer: The late fine is Rs. 2 per book for each overdue day.

Question: What are the library timings?

Answer: The library is open Monday to Saturday from 9:00 AM to 6:00 PM.

Question: Who can get membership?

Answer: Students with a valid student ID can become members.

Question: Is membership free?

Answer: Yes. Membership is free for enrolled students.

Sample Execution

Ask a library question: What is the late fine?

Retrieved Context:

Fines: The late fine is Rs. 2 per book for each overdue day.

Answer:

The late fine is Rs. 2 per book for each overdue day.

Library Information RAG – Result and Conclusion

Result

The Library Information RAG System successfully retrieves library rules relevant to a user's question and displays the retrieved information as an answer. The knowledge base covers the required areas of borrowing, renewal, fines, timings, and membership.

RAG Workflow

1. User enters a library question.
2. The question is converted into searchable words.
3. The system compares the question with the library knowledge base.
4. A relevance score is calculated for each rule.
5. The highest-scoring rule is retrieved.
6. The retrieved rule is displayed as context and answer.

Overall Conclusion

Both demonstrations show the fundamental idea of Retrieval-Augmented Generation. The Top-K system demonstrates how different values of K change the amount of retrieved context, while the Library Information RAG system applies retrieval to a practical question-answering task. The implementations are small, readable, and suitable for execution in Python IDLE or VS Code. The examples also show that retrieval quality is important because the final response depends on the context selected from the knowledge base.

Final Output

Top-1, Top-3, and Top-5 retrieval can be compared directly in the first program. The second program answers library questions using retrieved rules. Therefore, the required Top-K Retrieval Demonstration and Library Information RAG System are implemented successfully.